

4C16 Lab System Handbook

Vibhoothi, Nils Peters, François Pitié

Trinity College Dublin

2026/2027

Foreword

When we designed this module in 2017, we knew that labs would require expensive computing resources and that software frameworks would change very quickly.

We have thus designed a unique lab system, specifically for this course, so that you can work, remotely, in a modern environment, with up-to-date machines, learn good industry practices and get immediate feedback.

Labs are written in Python, using the PyTorch Deep Learning framework, running on Google's Colab notebooks, which will allow you to write and execute code through the browser. Assessments will be automatically graded through our git-based web application.

Why Python?

Python offers the convenience of high-level facilities for general programming, combined with specialised high-performance libraries (which are not themselves written in Python) for number crunching.

Also:

- ▶ Free
- ▶ Reasonably friendly
- ▶ Reasonably strict
- ▶ Can be nearly industrial
- ▶ (alas?) not Matlab

Not a good language for implementing high-performance cutting-edge algorithms, but a good language for using them.

Why Python?

Worldwide, Sept 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	52.08 %	+22.7 %
2		Java	13.34 %	-0.7 %
3		C/C++	7.99 %	-3.5 %
4	↑	R	3.83 %	-2.1 %
5	↓	JavaScript	3.28 %	-3.1 %
6	↑↑↑↑↑↑↑	Objective-C	3.21 %	+1.2 %
7		PHP	1.98 %	-1.5 %
8	↑	Rust	1.7 %	-1.1 %
9	↓↓↓	C#	1.6 %	-2.7 %
10	↓↓	Swift	1.54 %	-1.8 %
11	↓	Ada	1.47 %	-1.0 %
12	↓	TypeScript	1.43 %	-0.8 %
13		Matlab	0.9 %	-0.7 %
14	↑↑↑	Ruby	0.73 %	-0.3 %

source: The PYPL Popularity of Programming Language Index is created by analysing how often language tutorials are searched on Google. <http://pypl.github.io>

Why Python?

Python has become the standard for deep learning practitioners, with **R** being the language of choice for statisticians.



source: indeed.com (2017)

Also, let's not forget about **Rust**, that is seeing surging demand for performance-critical systems.

The 4C16 Lab System

Overview: Main Components

Under the hood, the 4C16 lab system is a fairly complex, there are 4 components that you will need to be familiar with:

- ▶ the [dashboard](https://4c16.dev/) at <https://4c16.dev/>, where you can manage your account and monitor your progress.
- ▶ the [git server](#) on 4c16.dev, hosts:
 - ▶ a git repository (Forgejo) to back up your files
 - ▶ an assessment repository to grade your labs
- ▶ your Trinity [Google drive](#), which allows you to access files, and configuration settings when running a Colab machine.
- ▶ [Colab](#), which will run a new virtual machine every time you start a Colab notebook.

Overview: Main Steps

Here is a quick overview of the steps you will have to follow. More details instructions follow in the coming slides.

The first time around, you will need to do two steps:

1. Change default password on 4c16.dev and get your private key information.
2. Open `4c16-init.ipynb` in Colab, paste the information, and run the cells to setup your google drive.

On daily usage, you will run in parallel 2 Colab notebooks:

1. `4c16-git-workflow.ipynb`, for fetching code on your google drive, pushing to 4c16.dev, and submitting labs for grading.
2. the current lab's Colab notebook instructions, found in your google drive, (eg. `4c16-labs/code/dist/lab-00/Lab 0.ipynb`).

Initialisation — Detailed Instructions

Step 0: Install Colaboratory

Adding Colab to your Google Drive

Your first step will be to add the Colaboratory application to your Google Drive:

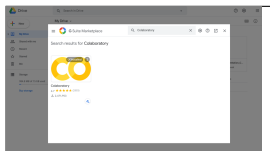
Go to your Google Drive settings:



Select Manage apps > Connect more apps:



Search and install Colaboratory:



Step 0: Colab and Google Drive Warning

Google Drive and Colab are free services offered by Google. This means that Google closely monitors their usage.

This especially applies to GPU instances (that we will be using from lab 4), but in general, be careful, don't have too many tabs open, don't mine crypto, don't abuse the system or Google might cripple your account.

Google doesn't make their rules public, but if you do the labs in the way they have been intended, you should be fine.

Step 1: <https://4c16.dev> (1/2)

log-in

- ▶ open a new browser
- ▶ go to <https://4c16.dev>
- ▶ login with your tcd email and password
- ▶ change your password!
- ▶ authorize "4c16-portal" to access your account.

Step 1: <https://4c16.dev> (2/2)

copy the init and workflow notebooks

Follow the **Init Colab Instructions** (click on instructions if they don't show)

Copy the Personalised configuration cell:

Personalised configuration Copy

```
module_org = "4c16-2026-27"  
username = "fp"  
real_name = "F P"  
private_key_b64 = "  
code_ssh_url = "ssh://git@4c16.dev:2222/4c16-2026-27/fp-code.git"  
labs_ssh_url = "ssh://git@4c16.dev:2222/4c16-2026-27/labs-source-dist.git"
```

Open the Init Colab Notebook:

Initial setup

Configure your private course repository and Forgejo connection. This needs to be done only once, at the start of term.

[Start here](#)

[Open init colab notebook ↗](#)

[Instructions](#)

The notebook is read-only, so you need to save a copy to your Google Drive (File > Save). By default these will be saved in your 'Colab Notebooks' folder.

Step 2: 4c16-init-2026-v1.ipynb (1/2)

once-off installation: setting up your Google Drive

You need to know that every time you open a new Colab notebook, Google will spin a **new** machine for you. This machine has a fresh installation by Google, and knows nothing about your files, so it will need to communicate with something. The way we resolve this is by mounting your Google Drive in your Colab runtime.

Your Google Drive is thus used to share lab material with your Colab machines. Because your Google Drive might contain personal data that you care about, Google will ask you for authorisation, etc.

We must now prepare our Google Drive and install a few things for our labs. This only needs to be done once at the beginning of the term.

Step 2: 4c16-init-2026-v1.ipynb (2/2)

Notebook Instructions:

In the first cell, copy the information about your username and private private keys.

Then evaluate all the cells.

Daily Usage — Detailed Instructions

4c16-git-workflow.ipynb (1/2)

daily git workflow

The lab system is reliant on a git server installed on 4c16.dev. This git repository saves your lab files and also allows you to submit your labs and get an automatic evaluation.

Ideally, we would be handling this with a terminal and use the command line commands such as `git clone`, `git commit`, `git push`, etc. (see brief git intro at end of slide deck). But the free-tier version of Colab doesn't feature a web terminal.

Thus, at each session, you will need to use the dedicated 4c16-git-workflow.ipynb notebook to manage your git workflow.

Daily Workflow

DAILY USE

Pull released labs, work in your repository, submit, and check feedback from one notebook.

Open daily workflow Colab ↗

4c16-git-workflow.ipynb (2/3)

daily git workflow

Open link to `4c16-git-workflow.ipynb` notebook provided to you on `4c16.dev`, make a copy to your Google Drive. You will be using this notebook to manage your daily git commands.

Note: this notebook must always be open before the lab's notebook.

This is because you need it to fetch a clean version of your lab material, which also includes the lab notebook.

4c16-git-workflow.ipynb (2/3)

daily git workflow

We will make a clean clone of your lab directories at the start of every session. So if your edits that haven't been updated to the remote server on 4c16.dev, they might simply disappear the next time you log in (no kidding).

**If you don't commit and push,
your code doesn't exist!**

(Couldn't write it bolder or redder, so you're on your own now).

4c16-git-workflow.ipynb (3/3)

The first cells are used to checkout lab material in your Google Drive (you must always run this cell first, before starting work on a lab)

The notebook also contains a cell for you to save your current work (commit and push). Run this cell regularly. Your work will be lost otherwise.

The Step 5 of the notebook pushes your work to the grading system on 4c16.dev. Read the output printout to see which tests you failed or passed.

Git Workflow Example

Getting Started with Lab 0

1. Open your `4c16-git-workflow.ipynb` notebook from your Google Drive. (always do this first)
2. Set `lab_id = 0` in first cell, evaluate next cell.
3. In google drive, go to
`MyDrive/tcd-4c16-labs/code/dist/lab-00`
4. Open `Lab 0.iynb`, read it and follow the instructions.
 - ▶ every time a chunk of work has been done, use `4c16-git-workflow.ipynb` to commit/push
 - ▶ every time an exercise is done, submit the lab for grading using `4c16-git-workflow.ipynb`

SCM

Git overview

It is essential to use source code management (version control) for any serious programming.

We use the Git SCM system — but we will only use some of the features (after all, we are not working collaboratively), and all you will be using is packaged in the 4c16-git-workflow.ipynb notebook.

Here is a brief overview of what we use:

- ▶ Initialise a *repository* (or *repo*)
- ▶ Make some modifications to your code
- ▶ Review and modifications and decide which to *stage*
- ▶ *Commit* stage modifications
- ▶ *Push* to a repo (back up/collaborate)

Git workflow

The git workflow is to prepare a copy, or image, of your work that represents a step forward, and then *commit* that copy to the record. Example:

```
git add testfile.py
git status
git commit -m "[lab-00] added test file"
```

Here we have added the file testfile.py to the image for committing, checked that the repository is in the state we expect (via git status), and then committed our changes

Git workflow

After editing a file, you can examine the modification with

```
git diff testfile.py
```

We can also commit all modifications with

```
git commit -a -m "Updated plenty of files"
```

Update the remote (permanent backup) copy of your repo with:

```
git push origin main
```

In our system, we also provide a git subcommand to trigger official grading on 4c16.dev:

```
git submit-lab {lab_id}
```

Credits

Vibhoothi [Lab Infrastructure], Research Fellow, working on optimised transcoding with AV1 and other codecs. He is also part of various open-source initiatives and projects, including Alliance For Open-Media (AOM), Xiph. Org, VideoLAN.

François Pitié [Labs], Assistant Professor in Media Signal Processing in the School of Engineering and part of the ADAPT Research Centre. His algorithms have been used by Google, Disney, Foundry, Weta Digital, and in many film productions.

Hugh Denman [Original Lab Infrastructure], PhD, former engineer at Google and YouTube. His code has been used to transcode 400 hours of video/min on YouTube since 2012. Hugh wrote the initial platform in 2017.

FAQ (1)

Q: Pushing for assessment outputs: Everything up-to-date

A: Have you committed and pushed to main beforehand?

Q: How do I edit (non-notebook) lab files?

A: If you use the desktop Google Drive application, you can open the files using any file editor; otherwise, you can use the ones provided by Google Drive, or you can also open the files through Colab itself. There is indeed a file manager on the left tab.

